Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу «Операционные системы»

Группа: М8О-210БВ-25
Студент: Верижников Владислав Павлович
Преподаватель: Бахарев В.Д.
Оценка: _____
Дата: 10.10.26

Москва, 2026

Постановка задачи

Вариант 5

Цель работы — приобрести практические навыки управления процессами и обмена данными между процессами посредством каналов.

Родительский процесс создаёт дочерний процесс. Первой строкой пользователь вводит имя файла, которое передаётся при создании дочернего процесса. Родитель и дочерний процесс представлены разными программами. Команды пользователя передаются через pipe1, связанный со стандартным входом дочернего процесса. Дочерний процесс при необходимости передаёт данные родителю через pipe2. Результаты работы дочерний процесс записывает в созданный им файл.

Пользователь вводит команды вида «число<endline>». Дочерний процесс проверяет число на простоту. Если число составное, оно записывается в файл. Если число отрицательное или простое, дочерний и родительский процессы завершаются.

Решение реализовано на C11 для ОС Linux с использованием системных интерфейсов. Ввод-вывод стандартной библиотеки C, threads.h, stdatomic.h и стандартная библиотека C++ не используются.

Общий метод и алгоритм решения

Родитель создаёт два безымянных канала вызовом pipe, затем создаёт дочерний процесс через fork. В ребёнке dup2 перенаправляет стандартный ввод в pipe1, стандартный вывод — в pipe2. Затем execv загружает отдельную программу child, которой аргументом передаётся имя файла. Ребёнок открывает файл через open и записывает в него составные числа через write.

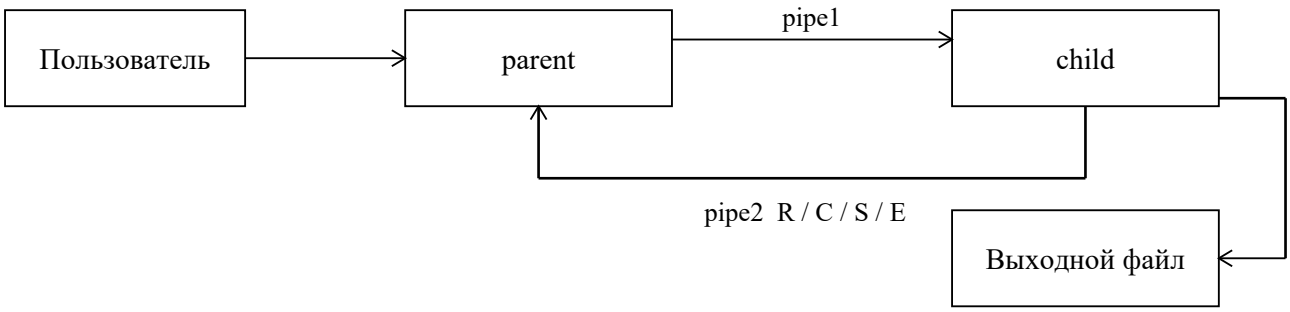


Схема взаимодействия процессов

После успешного открытия файла ребёнок отправляет ответ R. Родитель пересылает одну строку и ждёт однобайтовый ответ: C — обработка завершена, S — условие остановки, E — ошибка. Ответ после каждой команды позволяет родителю завершиться сразу после простого или отрицательного числа, не ожидая следующего ввода пользователя.

Ребёнок разбирает одно знаковое 32-битное целое число без scanf и atoi. Знак и окружающие ASCII-пробелы допускаются. Число накапливается в 64-битной переменной; после каждой цифры проверяется предел 2147483648, а после применения знака — диапазон int32. Для проверки простоты отдельно рассматриваются числа меньше 2; затем проверяются все делители d от 2 при $d \le n / d$. Условие не вызывает переполнения, в отличие от вычисления $d \times d$.

Числа 0 и 1 не являются простыми или составными, поэтому не записываются в файл и не завершают программу. Это принятое уточнение неописанного граничного случая. Некорректная команда завершает оба процесса с кодом 1; уже записанные результаты сохраняются. Конец входа или дополнительная команда exit завершают работу штатно.

Для простоты строка хранится в массиве на 4096 байт. Допускается до 4095 байт перед LF; слишком длинная строка отклоняется. write вызывается до передачи всех байтов, а read, write и waitpid повторяются при EINTR. Неиспользуемые концы каналов закрываются; EOF передаётся ребёнку закрытием записи в pipe1. Родитель ожидает ребёнка через waitpid и проверяет статус завершения. SIGPIPE игнорируется через sigaction, поэтому разрыв канала обрабатывается как ошибка записи.

Использованные системные вызовы

pipe — создаёт два канала для команд и ответов.

fork — создаёт дочерний процесс; execv — загружает программу child с именем выходного файла в аргументах.

dup2 — связывает стандартные дескрипторы ребёнка с каналами.

open — создаёт или перезаписывает выходной файл; read и write — читают команды, передают ответы и записывают составные числа.

close — закрывает ненужные файловые дескрипторы; waitpid — ожидает завершения ребёнка; _exit — завершает процесс.

sigaction — задаёт обработку SIGPIPE; readlink — определяет расположение родителя через /proc/self/exe для поиска программы child рядом с ним.

Ошибки системных операций проверяются. Диагностика выводится в stderr с названием операции и числовым errno. Код 0 означает штатное завершение, 1 — ошибку. Функции стандартной библиотеки для ввода-вывода и разбора чисел не используются.

Код программы

Полные исходные тексты совпадают с файлами src/common.h, src/parent.c и src/child.c в репозитории. Печать длинных строк в листинге может переноситься.

src/common.h

```
#ifndef LAB_COMMON_H
#define LAB_COMMON_H
#include <unistd.h>
#include <errno.h>
#include <stdint.h>
#include <stddef.h>
typedef struct {
    char data[4096];
    size_t size;
} Line;
static inline int write_all(int fd, const char *data, size_t size) {
    while (size > 0) {
        ssize_t n = write(fd, data, size);
        if (n < 0 && errno == EINTR) continue;
        if (n <= 0) return 0;
        data += n;
        size -= (size_t)n;
    }
    return 1;
}
static inline void message(const char *text) {
    size_t size = 0;
    while (text[size]) ++size;
    (void)write_all(STDERR_FILENO, text, size);
}
static inline size_t number_text(uint32_t number, char *text) {
    char reverse[10];
    size_t size = 0;
    do {
        reverse[size++] = (char)('0' + number % 10);
        number /= 10;
    } while (number);
    for (size_t i = 0; i < size; ++i) text[i] = reverse[size - i - 1];
    return size;
}
static inline void system_error(const char *operation) {
    int code = errno;
    char text[10];
    size_t size = number_text((uint32_t)code, text);
    message(operation);
    message(": errno ");
    (void)write_all(STDERR_FILENO, text, size);
    message("\n");
}
static inline int read_line(int fd, Line *line) {
    line->size = 0;
    for (;;) {
        char ch;
        ssize_t n = read(fd, &ch, 1);
        if (n < 0 && errno == EINTR) continue;
        if (n < 0) return -1;
        if (n == 0) break;
        if (ch == '\n') {
            if (line->size && line->data[line->size - 1] == '\r')
                --line->size;
            line->data[line->size] = 0;
            return 1;
        }
```

```
        if (line->size == sizeof(line->data) - 1) {
            errno = EMSGSIZE;
            return -1;
        }
        line->data[line->size++] = ch;
    }
    line->data[line->size] = 0;
    return line->size ? 1 : 0;
}
static inline int whitespace(char ch) {
    return ch == ' ' || ch == '\t' || ch == '\r' || ch == '\v' || ch == '\f';
}
static inline int parse_integer(const Line *line, int32_t *value) {
    size_t i = 0;
    int sign = 1;
    int64_t number = 0;
    while (i < line->size && whitespace(line->data[i])) ++i;
    if (i < line->size && (line->data[i] == '+' || line->data[i] == '-')) {
        if (line->data[i] == '-') sign = -1;
        ++i;
    }
    size_t start = i;
    while (i < line->size && line->data[i] >= '0' && line->data[i] <= '9') {
        number = number * 10 + line->data[i++] - '0';
        if (number > 2147483648LL) return 0;
    }
    if (i == start) return 0;
    while (i < line->size && whitespace(line->data[i])) ++i;
    number *= sign;
    if (i != line->size || number < INT32_MIN || number > INT32_MAX) return 0;
    *value = (int32_t)number;
    return 1;
}
static inline int is_prime(int32_t number) {
    if (number < 2) return 0;
    for (int32_t d = 2; d <= number / d; ++d)
        if (number % d == 0) return 0;
    return 1;
}
static inline void close_fd(int *fd) {
    if (*fd >= 0 && close(*fd) < 0) system_error("close");
    *fd = -1;
}
#endif
```

src/parent.c

```
#define _POSIX_C_SOURCE 200809L
#include "common.h"
#include <signal.h>
#include <sys/wait.h>
static int get_reply(int fd, char *reply) {
    ssize_t n;
    do {
        n = read(fd, reply, 1);
    } while (n < 0 && errno == EINTR);
    if (n < 0) system_error("read reply");
    else if (n == 0) message("Child closed the reply pipe.\n");
    return n == 1;
}
int main(void) {
    int to_child[2] = {-1, -1}, from_child[2] = {-1, -1};
    int result = 1, state, status;
    pid_t child = -1;
    Line line;
    char filename[4096], path[4096], reply;
    struct sigaction action = {0};
    action.sa_handler = SIG_IGN;
    if (sigaction(SIGPIPE, &action, NULL) < 0) {
        system_error("sigaction");
        goto finish;
    }
    message("Output filename, then one integer per line.\n");
    message("Prime/negative, exit or EOF stops the program.\n");
    state = read_line(STDIN_FILENO, &line);
    if (state < 0) {
        system_error("read filename");
        goto finish;
    }
    if (state == 0 || line.size == 0) {
        message("Filename is required.\n");
        goto finish;
    }
    for (size_t i = 0; i < line.size; ++i) {
        if (!line.data[i]) {
            message("Invalid filename.\n");
            goto finish;
        }
        filename[i] = line.data[i];
    }
    filename[line.size] = 0;
    ssize_t size = readlink("/proc/self/exe", path, sizeof(path) - 1);
    if (size < 0) {
        system_error("readlink");
        goto finish;
    }
    if ((size_t)size == sizeof(path) - 1) {
        message("Executable path is too long.\n");
        goto finish;
    }
    while (size > 0 && path[size - 1] != '/') --size;
    if ((size_t)size + 6 > sizeof(path)) {
        message("Executable path is too long.\n");
        goto finish;
    }
    const char name[] = "child";
    for (size_t i = 0; i < sizeof(name); ++i) path[size + (ssize_t)i] = name[i];
    if (pipe(to_child) < 0 || pipe(from_child) < 0) {
        system_error("pipe");
        goto finish;
    }
    child = fork();
```

```
    if (child < 0) {
        system_error("fork");
        goto finish;
    }
    if (child == 0) {
        if (dup2(to_child[0], STDIN_FILENO) < 0 ||
            dup2(from_child[1], STDOUT_FILENO) < 0) {
            system_error("dup2");
            _exit(1);
        }
        for (int i = 0; i < 2; ++i) {
            if (to_child[i] > STDERR_FILENO) close_fd(&to_child[i]);
            if (from_child[i] > STDERR_FILENO) close_fd(&from_child[i]);
        }
        char *args[] = {path, filename, NULL};
        execv(path, args);
        system_error("execv");
        _exit(1);
    }
    close_fd(&to_child[0]);
    close_fd(&from_child[1]);
    if (!get_reply(from_child[0], &reply) || reply != 'R') goto finish;
    while ((state = read_line(STDIN_FILENO, &line)) == 1) {
        if (line.size == 4 && line.data[0] == 'e' && line.data[1] == 'x' &&
            line.data[2] == 'i' && line.data[3] == 't') {
            result = 0;
            goto finish;
        }
        if (!write_all(to_child[1], line.data, line.size) ||
            !write_all(to_child[1], "\n", 1)) {
            system_error("write command");
            goto finish;
        }
        if (!get_reply(from_child[0], &reply)) goto finish;
        if (reply == 'S') {
            result = 0;
            goto finish;
        }
        if (reply != 'C') goto finish;
    }
    if (state < 0) system_error("read input");
    else result = 0;
finish:
    for (int i = 0; i < 2; ++i) {
        close_fd(&to_child[i]);
        close_fd(&from_child[i]);
    }
    if (child > 0) {
        pid_t waited;
        do {
            waited = waitpid(child, &status, 0);
        } while (waited < 0 && errno == EINTR);
        if (waited < 0) {
            system_error("waitpid");
            result = 1;
        } else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) result = 1;
    }
    _exit(result);
}
```

src/child.c

```
#include "common.h"
#include <fcntl.h>
int main(int argc, char **argv) {
    int file = -1, result = 1, state;
    Line line;
    if (argc != 2) {
        message("Usage: child OUTPUT_FILE\n");
        goto finish;
    }
    file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file < 0) {
        system_error("open");
        goto finish;
    }
    if (!write_all(STDOUT_FILENO, "R", 1)) {
        system_error("write reply");
        goto finish;
    }
    while ((state = read_line(STDIN_FILENO, &line)) == 1) {
        int32_t number;
        if (!parse_integer(&line, &number)) {
            message("Expected one signed 32-bit integer per line.\n");
            goto finish;
        }
        if (number < 0 || is_prime(number)) {
            if (!write_all(STDOUT_FILENO, "S", 1)) {
                system_error("write stop");
                goto finish;
            }
            result = 0;
            goto finish;
        }
        if (number > 1) {
            char text[11];
            size_t size = number_text((uint32_t)number, text);
            text[size++] = '\n';
            if (!write_all(file, text, size)) {
                system_error("write file");
                goto finish;
            }
        }
        if (!write_all(STDOUT_FILENO, "C", 1)) {
            system_error("write reply");
            goto finish;
        }
    }
    if (state < 0) system_error("read");
    else result = 0;
finish:
    if (result) (void)write_all(STDOUT_FILENO, "E", 1);
    if (file >= 0 && close(file) < 0) {
        system_error("close file");
        result = 1;
    }
    _exit(result);
}
```

Протокол работы программы

Проверка выполнена 10.10.2026 в Ubuntu 24.04.5 LTS под WSL 2. Использованы GCC 13.3.0 и Python 3.12.3. Обе программы собраны с флагами -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 без предупреждений. Все 23 интеграционных теста завершились успешно.

Сборка и проверка в Linux

```
make
make test
```

Запуск и результат

```
printf 'result.txt\n4\n6\n9\n7\n8\n' | ./build/parent
cat result.txt
```

Проверенное содержимое выходного файла; код завершения 0:

```
4
6
9
```

Число 7 является простым, поэтому оно не записывается, обе программы завершаются, а следующая строка 8 не обрабатывается.

Граничные случаи

Для входа -1, 4 получен пустой файл и код 0. Для 0, 1, 4 и EOF получена одна строка 4 и код 0. Число 2147483646 записывается как составное; 2147483647 завершает работу как простое. Последняя строка без LF также обрабатывается.

Проверка ошибок

Для 4, hello, 6 получены файл с одной строкой 4, сообщение Expected one signed 32-bit integer per line. и код 1. При отсутствии каталога проверяется ошибка open; при отсутствии программы child — ошибка execv. Переполнение int32, нулевой байт в команде и строка длиннее буфера завершили работу с кодом 1.

Интеграционные тесты

Набор tests/test_lab.py проверяет 23 сценария, сравнивает содержимое файла, код завершения родителя и задаёт таймаут для обнаружения зависания. Включены кириллица и пробелы в имени файла, допустимые в Linux кавычки в имени, перезапись файла и поток из 1000 составных чисел. Для запуска используется Python только как внешний инструмент проверки; сама программа написана на C.

Статус проверки

```
Linux version
Ubuntu 24.04.5 LTS, WSL 2
GCC 13.3.0, Python 3.12.3
Build flags: -std=c11 -Wall -Wextra -Wpedantic -Werror -O2
Build: PASS, no warnings
PASS composites followed by prime
PASS negative stops immediately
PASS prime 2 stops immediately
PASS zero and one are skipped
PASS signed INT_MIN
PASS INT_MAX is prime
PASS final line without newline
PASS CRLF spaces and plus sign
PASS empty command stream
PASS manual exit
```

```
PASS malformed input
PASS multiple integers rejected
PASS positive overflow
PASS negative overflow
PASS embedded NUL rejected
PASS empty line rejected
PASS too long line rejected
PASS UTF-8 path with spaces
PASS existing file is truncated
PASS unavailable output directory
PASS quotation in Linux filename
PASS 1000 composite numbers
PASS missing child executable
23/23 integration tests passed
```

Вывод

Подготовлено решение для Linux с созданием отдельной дочерней программы и обменом через два канала. Дочерний процесс проверяет числа на простоту, записывает составные числа в файл и уведомляет родителя об остановке. Предусмотрены проверка системных ошибок, переполнение числа и закрытие дескрипторов. Сборка без предупреждений и 23 успешных интеграционных теста подтвердили корректность проверенных сценариев в Linux.